Next.js 14 · Gemini + Groq · Neon Postgres · Deployed on Vercel

GapFinder

Don't just find the wrong answer — find the exact line where the student's reasoning stopped following, name the misconception behind it, and teach that one thing until it holds without help.

THE PROBLEM

Grading the destination

Every homework tool marks the final answer. A student already knew they got it wrong. Being shown a correct solution to compare against does not tell them which of their own moves was the bad one.

→

THE INSIGHT

Most wrong lines aren't mistakes

A student who errs at line two and writes six lines has five wrong lines. Four are faithful consequences of one broken idea. Marking all five teaches them they are bad at algebra. Finding the one teaches them algebra.

→

THE SOLUTION

Locate it, name it, close it

GapFinder verifies every line against the one above it, isolates the *first* divergence, names the misconception from a fixed catalogue, teaches it — then re-tests with the help taken away.

LIVE APPLICATION
gap-finder-green.vercel.app

REPOSITORY
github.com/sanikommu-bhanu/GapFinder

SCOPE
31 pages · 40 API routes · 30 data models

VERIFICATION
228 tests · 15-case reasoning eval

WHY THIS EXISTS

A wrong answer is a symptom. The gap is the diagnosis.

GapFinder is built on one claim that can be checked rather than believed: the useful unit of feedback is not the answer, it is **the first line that stopped following from the one above it**.

The worked example the product is built around

A student expands two brackets and subtracts. Line 2 drops the negative across the second bracket. Lines 3–6 are then *correct algebra applied to a wrong line*.

A CONVENTIONAL CHECKER SAYS

"Incorrect. The answer is x = ..."

Five lines are implicitly wrong. The student learns that the whole attempt failed.

GAPFINDER SAYS

"Line 2 is where it broke — you distributed the minus to the first term only."

One line is wrong. Four are consequences. One habit is named, and it is the habit that will reappear next week.

01 — WHO IT SERVES

The student mid-attempt

Designed for a secondary or early-undergraduate student working through Math, Physics, Chemistry or Biology problems on a phone — the moment between finishing an attempt and having no idea why it failed.

It also serves the blank page: **Solve With Me** runs the same engine forwards, naming the next move without ever writing the line.

02 — WHY IT MATTERS

Misconceptions are durable

An arithmetic slip is noise. A misconception is a rule the student is faithfully applying — it keeps producing errors across chapters until it is named and replaced.

A **fixed catalogue code** per error makes the same habit countable across weeks — which is what turns feedback into a trajectory.

03 — WHAT HAS BEEN BUILT

A complete, deployed loop

Not a prototype of one screen. The full path — submit, verify, diagnose, teach, practise, transfer, re-test under exam conditions, and remember — is implemented and running.

31 pages · 40 API routes · 30 models · 22 concepts · 58 knowledge chunks · 228 tests

The rule that governs the whole system: the AI interprets, the code verifies. A model reads the handwriting and phrases the teaching — but it is never the authority on whether a step is correct. That judgement is deterministic algebra, which is why the product can afford to be this specific.

THE GAPFINDER DIFFERENCE

Two loops, and only one of them closes

The conventional loop terminates at a verdict. GapFinder's loop terminates only when the repair has survived a test with the help removed — and then feeds what it learned into the next attempt.

TRADITIONAL LEARNING TOOL

Question → verdict → next question

01	Question	given
02	Answer	the destination only
03	Correct / wrong	a binary verdict
04	Next question	nothing carried forward

↳ The reasoning is never inspected, so the same misconception survives intact.

GAPFINDER

Work → gap → repair → proof → memory

01	Work	the whole attempt
02	Reasoning	each line as a claim
03	Gap	first divergence, proved
04	Explanation	grounded in curated sources
05	Practice	built for that misconception
06	Verification	transfer + exam, no hints
07	Mastery	scored from evidence
08	Next action	roadmap reordered

The difference is not more feedback — it is feedback with a location. "Wrong" tells a student to try harder. "Line 2, and here is the rule you were applying" tells them what to change. Only the second one can be practised, re-tested, and eventually closed.

PRODUCT ECOSYSTEM

Fourteen surfaces, one loop

Every feature listed is implemented and reachable in the running app. They are grouped by the job they do in the loop, not by where they sit in the menu.

01 Diagnosis

Turn an attempt into a named, evidenced gap.

Check My Work	Photograph or type working. The pipeline reads it, verifies every line, and locates the first divergence.	`/scan`
My Learning Gaps	Open gaps, an error fingerprint of recurring misconceptions, and per-concept mastery.	`/gaps`
Analysis & confirm	Step-by-step verdicts, plus a confirmation screen when handwriting was ambiguous.	`/analysis/[id]`

02 Learning

Teach the one thing that broke — or help before anything breaks.

Ask a Concept	Any question routed to a curated concept, answered with a computed diagram, a read-aloud lesson and cited sources.	`/learn`
Solve With Me	For a blank page. Names the next move and why — the app never writes the line for you.	`/solve`
AI Coach	Retrieval-grounded, and can see your own record. Says so honestly when the context doesn't cover a question.	`/coach`

03 Mastery

Prove the repair holds when the help is taken away.

Practice & Teach-back	Targeted problems, then explaining the idea back in your own words against a rubric.	`/gaps/[id]/…`
Transfer	The same concept in an unfamiliar problem — weighted double in mastery scoring.	`/gaps/[id]/transfer`
Exam Mode	No hints, no feedback. Never claims mastery from one question; thin evidence returns “uncertain”.	`/exam`
Focus Mode	A timed block around one problem, with optional Spotify playback.	`/focus`

04 Progress & trust

Make the learner's own trajectory — and the system's reasoning — visible.

Learning Roadmap	What to learn next, ordered by what your own work showed, with prerequisites locked.	`/roadmap`
History & Full Report	Every analysis by day with its outcome, and a consolidated report across all work.	`/history` · `/reports/full`
Achievements	Five milestones tied to real evidence — gaps found, repaired, transferred, taught back.	`/achievements`
AI Observability	Every model call for your analyses: stage, provider, cache status, latency, errors.	`/dev/observability`

The shared spine

All fourteen surfaces read and write the same learner state, so a gap found in Check My Work is the gap the Roadmap orders, the Coach can see, Exam Mode re-tests, and the next analysis already knows about.

Gap · MasteryRecord · LearningEvent · Roadmap · Analysis · misconception code

GapFinder

Diagram 04 — Product ecosystem

WHY IT IS TECHNICALLY INTERESTING

Six decisions that are not the obvious ones

Each of these is a place where the easy implementation would have been to ask a language model, and the system deliberately does something else.

01 · REASONING ANALYSIS

Correctness is decided by algebra, not by a model

Each step-pair is tested with `mathjs`: move both sides to one expression and check the transformation preserves the solution set. An LLM asked "is this step right?" is confidently wrong often enough to be useless for accusation.

02 · LEARNING-GAP MODELLING

A closed catalogue, not free text

17 misconception codes across four subjects, plus an explicit UNCLASSIFIED outcome. Where the algebraic signature is unambiguous the code is *derived*, so the same student error always yields the same code — making errors countable rather than merely describable.

03 · GRACEFUL DEGRADATION

The pipeline survives its own dependencies

Provider cascade — cache → Gemini → Groq → deterministic fallback. A free-tier rate limit is treated as a certainty, not an edge case. Every stage after reading a photo has a local substitute.

04 · GROUNDED GENERATION

Retrieval without a vector database

58 curated knowledge chunks scored by local keyword overlap. The corpus is small and hand-written, so the retrieval is free, local and auditable — and every explanation cites the chunk ids it used.

05 · PERSISTENT LEARNER STATE

Mastery is evidence, computed

A 0–100 EMA per concept ($\alpha = 0.35$). Transfer success is weighted +12 against practice's +6, because solving the same idea in an unfamiliar problem is the stronger evidence. No model votes on this.

06 · OBSERVABILITY

Every model call is inspectable

Stage, `provider:model`, cache status, latency, error text and retrieved chunk ids are written per call, and surfaced in a per-analysis trace view — so "what did the AI actually do here?" has an answer.

THE ONE-SCREEN SUMMARY

If you read nothing else in this dossier, this page is the project.

GapFinder in a single frame

THE CLAIM

A mistake is not a score. It is a location, a name, and a next action.

GapFinder finds the first line that stopped following, names the misconception behind it from a fixed catalogue, teaches that one thing, and refuses to call it mastered until the student can do it again with the help removed.

THE GOVERNING RULE

The AI interprets.
The code verifies.
The database remembers.

HOW IT WORKS

- **Read** — one multimodal call turns handwriting into lines, or the student types them.
- **Verify** — every transition checked by `mathjs`; algebra, quantitative and chemical domains routed per step.
- **Locate** — the *first* failure is the gap; later failures are its consequences.
- **Name** — one of 17 catalogue codes, proved where the signature is unambiguous.
- **Teach** — explanation grounded in curated chunks, with a computed diagram.
- **Prove** — practice, teach-back, transfer, then Exam Mode with no hints.
- **Remember** — mastery EMA, gap status, roadmap order, recurring codes.

RESILIENCE BY DESIGN

Classification and explanation both fall back to deterministic logic over the same verified algebra. Only photo-reading has no local substitute — and the app offers typed working, which runs the identical pipeline with no AI at all.

WHAT IS ACTUALLY BUILT

31	40	30
pages	API routes	data models
22	**58**	**17**
seeded concepts across 4 subjects	curated knowledge chunks	misconception codes

PROVEN, ON ANY MACHINE

Unit tests	**228 passing** across 14 files
Reasoning eval	**13 of 13** scored cases pass, 2 skipped — 100% divergence accuracy
Build	Lint and typecheck clean; build succeeds
Deployment	Live on Vercel, responding